

LABORATORIO N° 02

Diseño y Desarrollo de Simuladores y Videojuegos



DOCENTE:

Bestard Aroche, Yunion

CURSO:

Desarrollo de Aplicaciones
Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

Identificar y formalizar los requisitos de una problemática real (tienda, biblioteca, reservas, etc.) como base para el diseño de una aplicación Django.

Diseñar e implementar una nueva App Django (Model, View, URL, Template) conectada a un proyecto existente, aplicando el patrón MVT.

Implementar formularios (Forms) que traduzcan los requisitos identificados en una funcionalidad real de captura y persistencia de datos.

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

Introducción al laboratorio

En las sesiones anteriores construimos y extendimos el proyecto DjangoInicial, comprendiendo cómo Django organiza una aplicación mediante el patrón MVT: Project, Apps, Models, Views, URLs y Templates. En este laboratorio damos un paso más cercano a cómo se trabaja en un proyecto real: en lugar de partir de un modelo ya definido, ustedes van a investigar una problemática real, capturar sus requisitos, y traducir esos requisitos en una aplicación Django funcional, construida como una nueva App conectada al proyecto que ya conocen (config/core).

Para mantener el foco en el flujo MVT y en cómo un formulario transforma datos en un response, esta aplicación no usará base de datos: los datos se definirán de forma estática dentro de

models.py (por ejemplo, como una lista de diccionarios), y las vistas leerán y agregarán información directamente sobre esa estructura en memoria, sin migraciones ni panel de administración.

Pueden elegir cualquier problemática cotidiana que se preste a una aplicación sencilla: tienda de barrio, biblioteca, veterinaria, alquiler de equipos, reserva de citas, control de asistencia, u otra que les resulte relevante.

Enunciados

Ejercicio 1 — Investigar una problemática real

Investiguen una problemática cotidiana que pueda resolverse con una aplicación web (por ejemplo: tienda de barrio, biblioteca, veterinaria, alquiler de equipos, reserva de citas, control de asistencia, u otra de su interés). Redacten en 3-5 líneas en qué consiste el problema y quién lo usaría.

Problemática:

Una pizzería artesanal presenta problemas operativos en dos áreas clave: por un lado, cambia constantemente su menú según los insumos disponibles pero no lleva un registro claro de las especialidades activas; por otro lado, toma los pedidos de los clientes en papel, lo que genera desorden en la cocina y retrasos en las entregas a domicilio.

Ejercicio 2 — Capturar los requisitos

A partir de la problemática elegida, listen entre 4 y 6 requisitos funcionales del sistema (ejemplo: "el sistema debe permitir registrar un producto", "el usuario debe poder ver el listado de productos disponibles"). Cada requisito debe redactarse como una acción concreta que el sistema debe permitir.

- El sistema debe mostrar el listado con todas las especialidades de pizza registradas y sus ingredientes principales.
- El sistema debe proveer un formulario web para registrar nuevas pizzas en la lista del menú.
- El sistema debe registrar obligatoriamente el nombre, tipo de masa, ingredientes y el estado del stock de insumos.
- El sistema debe actualizar el menú en pantalla inmediatamente después de procesar y guardar los datos.
- El sistema debe mostrar el listado de pedidos del día con su estado actual de preparación.
- El sistema debe contar con un formulario para registrar las nuevas solicitudes de los clientes.
- El sistema debe capturar el nombre del cliente, teléfono, pizza solicitada, dirección y estado del pedido.
- El sistema debe validar que todos los campos del pedido estén completos antes de agregarlo a la lista de trabajo de cocina.

Ejercicio 3 — Diseñar el modelo de datos

A partir de los requisitos del Ejercicio 2, identifiquen la entidad principal de su problemática (por ejemplo: Producto, Libro, Cita, Equipo) y definan sus campos (nombre, tipo de dato y si es obligatorio). Justifiquen brevemente por qué cada campo es necesario según los requisitos capturados.

1. Entidad: Menú (Pizza)

- **name:** Texto (CharField), Obligatorio: Sí, Permite identificar de manera única la especialidad de la pizza en el menú.
- **dough_type:** Selección (ChoiceField), Obligatorio: Sí, Especifica el tipo de masa (delgada, tradicional, integral, etc.) para la preparación.
- **ingredients:** Texto Largo (Textarea), Obligatorio: Sí, Describe los insumos que componen la pizza para información del cliente.
- **stock_status:** Selección (ChoiceField), Obligatorio: Sí, Indica la disponibilidad del producto (Disponible, Poco stock, Agotado).

2. Entidad: Pedidos (Order)

- **customer_name:** Texto (CharField), Obligatorio: Sí, Registra el nombre del cliente para identificar a quién pertenece la orden.
- **phone:** Texto (CharField), Obligatorio: Sí, Permite contactar al cliente en caso de inconvenientes con el delivery.
- **pizza_name:** Texto (CharField), Obligatorio: Sí, Almacena el nombre de la pizza solicitada por el usuario.
- **address:** Texto Largo (Textarea), Obligatorio: Sí, Dirección exacta donde la unidad de reparto entregará el pedido.
- **status:** Selección (ChoiceField), Obligatorio: Sí, Controla el estado del pedido (Pendiente, En preparación, Entregado, Cancelado).

Ejercicio 4 — Crear la nueva App

Dentro del proyecto base (config/core), creen una nueva App con un nombre representativo de su problemática (por ejemplo store, library, bookings). Regístrenla en INSTALLED_APPS.

```
33  INSTALLED_APPS = [  
34      'django.contrib.admin',  
35      'django.contrib.auth',  
36      'django.contrib.contenttypes',  
37      'django.contrib.sessions',  
38      'django.contrib.messages',  
39      'django.contrib.staticfiles',  
40      'core',  
41      'pizza',  
42      'order',  
43  ]
```

Ejercicio 5 — Implementar el Model con datos estáticos

Traduzcan el diseño del Ejercicio 3 en models.py, definiendo los datos como una lista de diccionarios (o de objetos de una clase simple) con al menos 5 registros de ejemplo. No se usan migraciones ni base de datos: esta lista es la única fuente de datos de la aplicación.

```
src > order > models.py > ...
```

```
1  ORDERS_DB = [  
2      {  
3          'customer_name': 'Ana López',  
4          'phone': '555-0101',  
5          'pizza_name': 'Margarita',  
6          'address': 'Av. Principal 123',  
7          'status': 'Pendiente',  
8      },  
9      {  
10         'customer_name': 'Bruno García',  
11         'phone': '555-0102',  
12         'pizza_name': 'Pepperoni',  
13         'address': 'Calle Los Olivos 456',  
14         'status': 'En preparación',  
15     },  
16     {  
17         'customer_name': 'Carla Silva',  
18         'phone': '555-0103',  
19         'pizza_name': 'Vegetariana',  
20         'address': 'Jr. Las Flores 789',  
21         'status': 'Entregado',  
22     },  
23     {  
24         'customer_name': 'Diego Pardo',  
25         'phone': '555-0104',  
26         'pizza_name': 'Cuatro Quesos',  
27         'address': 'Av. Central 321',  
28         'status': 'Pendiente',  
29     },  
30     {  
31         'customer_name': 'Eva Morales',  
32         'phone': '555-0105',  
33         'pizza_name': 'Hawayana',  
34         'address': 'Calle Primavera 654',  
35         'status': 'Cancelado',  
36     },  
37 ]
```

```
src > pizza > models.py > ...
```

```
1  PIZZAS_DB = [  
2      {  
3          'name': 'Margarita',  
4          'dough_type': 'Masa delgada',  
5          'ingredients': 'Tomate, queso mozzarella, albahaca',  
6          'stock_status': 'Disponible',  
7      },  
8      {  
9          'name': 'Pepperoni',  
10         'dough_type': 'Masa tradicional',  
11         'ingredients': 'Tomate, queso mozzarella, pepperoni',  
12         'stock_status': 'Disponible',  
13     },  
14     {  
15         'name': 'Vegetariana',  
16         'dough_type': 'Masa integral',  
17         'ingredients': 'Tomate, mozzarella, champiñones, pimientos, cebolla',  
18         'stock_status': 'Disponible',  
19     },  
20     {  
21         'name': 'Cuatro Quesos',  
22         'dough_type': 'Masa tradicional',  
23         'ingredients': 'Mozzarella, parmesano, gorgonzola, provolone',  
24         'stock_status': 'Poco stock',  
25     },  
26     {  
27         'name': 'Hawayana',  
28         'dough_type': 'Masa delgada',  
29         'ingredients': 'Tomate, queso mozzarella, jamón, piña',  
30         'stock_status': 'Agotado',  
31     },  
32 ]
```

Ejercicio 6 — Implementar el listado (View + URL + Template)

Crean una vista que recorra la lista definida en models.py y la muestre en un template que herede de base.html, siguiendo el mismo patrón usado en core.

```
src > pizza > views.py > pizza_create
1  from django.shortcuts import redirect, render
2
3  from .forms import PizzaForm
4  from .models import PIZZAS_DB
5
6
7  def pizza_list(request):
8      return render(request, 'pizza/pizza_list.html', {'pizzas': PIZZAS_DB})
9
10
11 def pizza_create(request):
12     form = PizzaForm(request.POST or None)
13     if request.method == 'POST' and form.is_valid():
14         PIZZAS_DB.append(form.cleaned_data)
15         return redirect('pizza_list')
16
17     return render(request, 'pizza/pizza_create.html', {'form': form})
```

```
src > pizza > urls.py > ...
1  from django.urls import path
2
3  from .views import pizza_create, pizza_list
4
5
6  urlpatterns = [
7      path('', pizza_list, name='pizza_list'),
8      path('create/', pizza_create, name='pizza_create'),
9  ]
```

```
src > pizza > templates > pizza > pizza_list.html > table > tbody
1  {% extends "core/base.html" %}
2
3  {% block title %}Menú de Pizzas{% endblock %}
4
5  {% block content %}
6      <h2>Menú de Pizzas Artesanales</h2>
7      <p><a href="{% url 'pizza_create' %}">Agregar Pizza</a> | <a href="{% url 'order_list' %}">Ver Pedidos</a></p>
8      <table>
9          <thead>
10             <tr>
11                 <th>Nombre</th>
12                 <th>Tipo de Masa</th>
13                 <th>Ingredientes</th>
14                 <th>Estado del Stock</th>
15             </tr>
16         </thead>
17         <tbody>
18             {% for pizza in pizzas %}
19                 <tr>
20                     <td>{{ pizza.name }}</td>
21                     <td>{{ pizza.dough_type }}</td>
22                     <td>{{ pizza.ingredients }}</td>
23                     <td>{{ pizza.stock_status }}</td>
24                 </tr>
25             {% endfor %}
26         </tbody>
27     </table>
28 {% endblock %}
```

src > order >  views.py > ...

```

1  from django.shortcuts import redirect, render
2
3  from .forms import OrderForm
4  from .models import ORDERS_DB
5
6
7  def order_list(request):
8      return render(request, 'order/order_list.html', {'orders': ORDERS_DB})
9
10
11 def order_create(request):
12     form = OrderForm(request.POST or None)
13     if request.method == 'POST' and form.is_valid():
14         ORDERS_DB.append(form.cleaned_data)
15         return redirect('order_list')
16
17     return render(request, 'order/order_create.html', {'form': form})

```

src > order >  urls.py > ...

```

1  from django.urls import path
2
3  from .views import order_create, order_list
4
5
6  urlpatterns = [
7      path('', order_list, name='order_list'),
8      path('create/', order_create, name='order_create'),
9  ]

```

src > order > templates > order >  order_list.html > ...

```

1  {% extends "core/base.html" %}
2
3  {% block title %}Lista de Pedidos{% endblock %}
4
5  {% block content %}
6      <h2>Pedidos de la Pizzeria</h2>
7      <p><a href="{% url 'order_create' %}">Registrar Pedido</a> | <a href="{% url 'pizza_list' %}">Ver Menú</a></p>
8      <table>
9          <thead>
10             <tr>
11                 <th>Cliente</th>
12                 <th>Teléfono</th>
13                 <th>Pizza</th>
14                 <th>Dirección</th>
15                 <th>Estado</th>
16             </tr>
17         </thead>
18         <tbody>
19             {% for order in orders %}
20                 <tr>
21                     <td>{{ order.customer_name }}</td>
22                     <td>{{ order.phone }}</td>
23                     <td>{{ order.pizza_name }}</td>
24                     <td>{{ order.address }}</td>
25                     <td>{{ order.status }}</td>
26                 </tr>
27             {% endfor %}
28         </tbody>
29     </table>
30 {% endblock %}

```


Ejercicio 7 — Implementar el formulario (Forms)

Crean un forms.py con un forms.Form (no ModelForm, ya que no hay modelo de base de datos) cuyos campos correspondan a los requisitos capturados en el Ejercicio 2.

```
src > pizza > forms.py > PizzaForm
```

```
1  from django import forms
2
3  MASA_CHOICES = [
4      ('Masa delgada', 'Masa delgada'),
5      ('Masa tradicional', 'Masa tradicional'),
6      ('Masa integral', 'Masa integral'),
7      ('Masa artesanal', 'Masa artesanal'),
8  ]
9
10 STOCK_CHOICES = [
11     ('Disponible', 'Disponible'),
12     ('Poco stock', 'Poco stock'),
13     ('Agotado', 'Agotado'),
14 ]
15
16
17 class PizzaForm(forms.Form):
18     name = forms.CharField(max_length=100, label='Nombre de la Pizza')
19     dough_type = forms.ChoiceField(
20         choices=MASA_CHOICES, label='Tipo de Masa', widget=forms.Select
21     )
22     ingredients = forms.CharField(widget=forms.Textarea, label='Ingredientes')
23     stock_status = forms.ChoiceField(
24         choices=STOCK_CHOICES, label='Estado del Stock', widget=forms.Select
25     )
```

```
src > order > forms.py > OrderForm
```

```
1  from django import forms
2
3  ESTADO_CHOICES = [
4      ('Pendiente', 'Pendiente'),
5      ('En preparación', 'En preparación'),
6      ('Entregado', 'Entregado'),
7      ('Cancelado', 'Cancelado'),
8  ]
9
10
11 class OrderForm(forms.Form):
12     customer_name = forms.CharField(max_length=100, label='Nombre del Cliente')
13     phone = forms.CharField(max_length=30, label='Teléfono')
14     pizza_name = forms.CharField(max_length=100, label='Pizza Solicitada')
15     address = forms.CharField(widget=forms.Textarea, label='Dirección de Envío')
16     status = forms.ChoiceField(
17         choices=ESTADO_CHOICES, label='Estado del Pedido', widget=forms.Select
18     )
```


Ejercicio 8 — Implementar la vista de creación

Crean una vista que muestre el formulario del Ejercicio 7, procese el envío (POST), valide los datos y agregue el nuevo registro a la lista en memoria definida en models.py. Luego redirijan al listado para confirmar que el nuevo dato aparece. Tengan en cuenta que, al no haber base de datos, los datos agregados se pierden al reiniciar el servidor — esto es esperado y debe mencionarse en su documentación.

← → ↻ http://127.0.0.1:8000/pizza/create/

Mi proyecto Django

Agregar Nueva Pizza

Nombre de la Pizza:

Tipo de Masa:

Ingredientes:

Estado del Stock:

[Volver al Menú](#)

← → ↻ http://127.0.0.1:8000/pizza/

Mi proyecto Django

Menú de Pizzas Artesanales

[Agregar Pizza](#) | [Ver Pedidos](#)

Nombre	Tipo de Masa	Ingredientes	Estado del Stock
Margarita	Masa delgada	Tomate, queso mozzarella, albahaca	Disponible
Pepperoni	Masa tradicional	Tomate, queso mozzarella, pepperoni	Disponible
Vegetariana	Masa integral	Tomate, mozzarella, champiñones, pimientos, cebolla	Disponible
Cuatro Quesos	Masa tradicional	Mozzarella, parmesano, gorgonzola, provolone	Poco stock
Hawayana	Masa delgada	Tomate, queso mozzarella, jamón, piña	Agotado
Pizza Peruana	Masa delgada	salsa de tomate, queso y hotdog	Disponible

← → ↻ http://127.0.0.1:8000/order/create/

Mi proyecto Django

Registrar Nuevo Pedido

Nombre del Cliente:

Teléfono:

Pizza Solicitada:

Dirección de Envío:

El agustino

Estado del Pedido:

[Volver a los Pedidos](#)

← → ↻ http://127.0.0.1:8000/order/

Mi proyecto Django

Pedidos de la Pizzería

[Registrar Pedido](#) | [Ver Menú](#)

Cliente	Teléfono	Pizza	Dirección	Estado
Ana López	555-0101	Margarita	Av. Principal 123	Pendiente
Bruno García	555-0102	Pepperoni	Calle Los Olivos 456	En preparación
Carla Silva	555-0103	Vegetariana	Jr. Las Flores 789	Entregado
Diego Pardo	555-0104	Cuatro Quesos	Av. Central 321	Pendiente
Eva Morales	555-0105	Hawayana	Calle Primavera 654	Cancelado
Joel	987654321	Pizza Peruana	El agustino	Pendiente

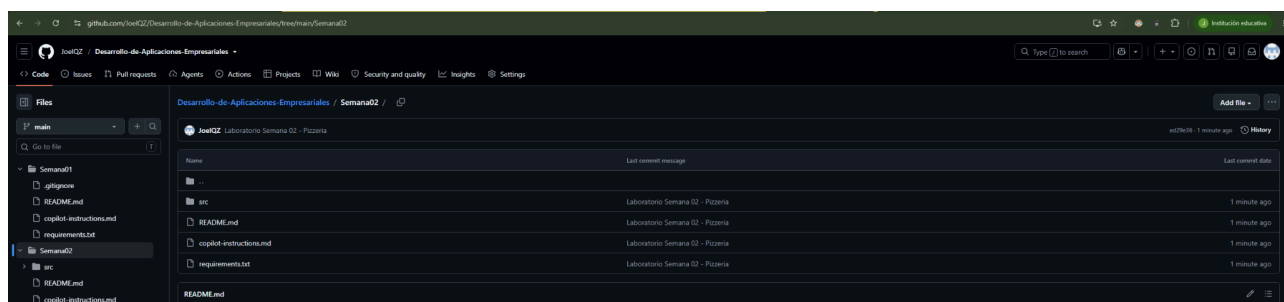
Ejercicio 9 — Verificación del flujo completo

Naveguen su aplicación de principio a fin (listado → crear → volver al listado con el nuevo dato reflejado) y documenten con capturas el recorrido Request → URL → View → Model (datos estáticos) → Template → Response. Expliquen cómo su nueva App convive con core dentro del mismo Project.

Se demuestra cómo funciona todo Django usando 4 capturas seguidas en el navegador que está en el ejercicio 8, primero entrando al listado, luego al formulario, después llenándolo con datos y finalmente viendo cómo se guardó el nuevo registro al redirigir al listado. Para explicarlo fácil en el informe, una petición (Request) nace cuando el usuario hace clic o envía datos; el sistema busca la dirección (URL) correcta y llama a la vista (View), la cual procesa los datos y los guarda agregándolos a la lista en memoria (Model); de ahí la vista le avisa a la plantilla (Template) para armar la página y devolvérsela lista al usuario (Response). Todo esto convive sin problemas con la carpeta principal (core) porque config/urls.py reparte las direcciones a cada app y las plantillas heredan el diseño base (base.html) para verse uniformes.

Ejercicio 10 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo su problemática, sus requisitos y la App creada. Hagan commit y push al repositorio del equipo.



<https://github.com/JoelQZ/Desarrollo-de-Aplicaciones-Empresariales/tree/main/Semana02>

Entregables

- Documento de requisitos: problemática elegida (Ejercicio 1) y lista de requisitos funcionales (Ejercicio 2).
- Diseño del modelo de datos: entidad principal, campos, tipos y justificación (Ejercicio 3).
- Código fuente completo de la nueva App: models.py (con los datos estáticos), views.py, urls.py, forms.py y templates (listado y formulario).
- Capturas de pantalla del flujo funcionando: listado de registros, formulario de creación, y el nuevo registro reflejado en el listado.
- Explicación del flujo MVT aplicado a su caso (Ejercicio 9), incluyendo cómo su App se conecta con core dentro del mismo Project.
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo, según el formato de evidencia indicado en las instrucciones del laboratorio (nombre, título, captura, código, explicación y casos de prueba).

Conclusiones:

En este laboratorio aprendimos a usar el patrón MVT de Django para conectar nuestras vistas, plantillas y rutas creando las apps de pizzas y pedidos sin usar base de datos, guardando la información en memoria RAM de forma limpia. También logramos unir estas nuevas aplicaciones con la estructura principal usando plantillas heredadas para que todo el sitio tenga el mismo diseño, y finalmente organizamos y subimos el proyecto a GitHub con su archivo de requisitos y documentación, dejando todo listo para trabajar en equipo.

Integrantes:

Joel Facundo Quijada Zevallos

Juan Diego Ramos Enriquez